

Large Language Models in AI

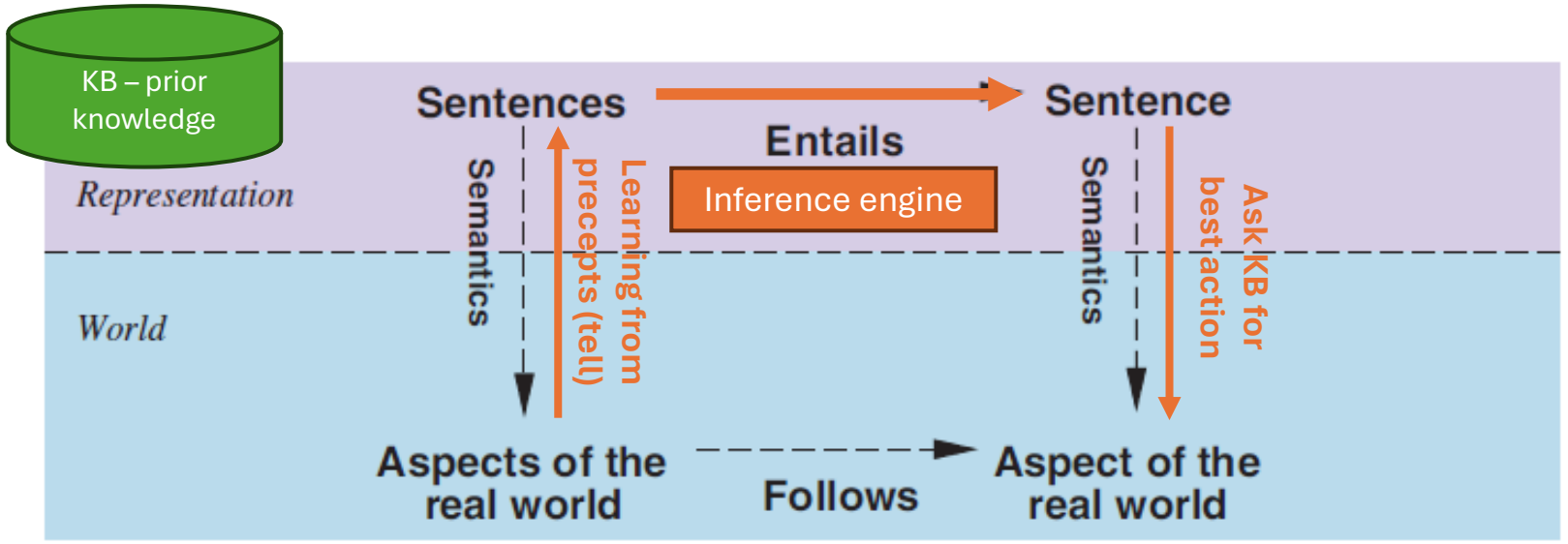
A Quick Overview

Michael Hahsler

Online Material

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.

Remember: Classical Knowledge-based Agents use Logic



function KB-AGENT(*percept*) **returns** an *action*
  persistent: *KB*, a knowledge base
    t, a counter, initially 0, indicating time

  TELL(*KB*, MAKE-PERCEPT-SENTENCE(*percept*, *t*))
  action ← ASK(*KB*, MAKE-ACTION-QUERY(*t*))
  TELL(*KB*, MAKE-ACTION-SENTENCE(*action*, *t*))
  t ← *t* + 1
  return *action*

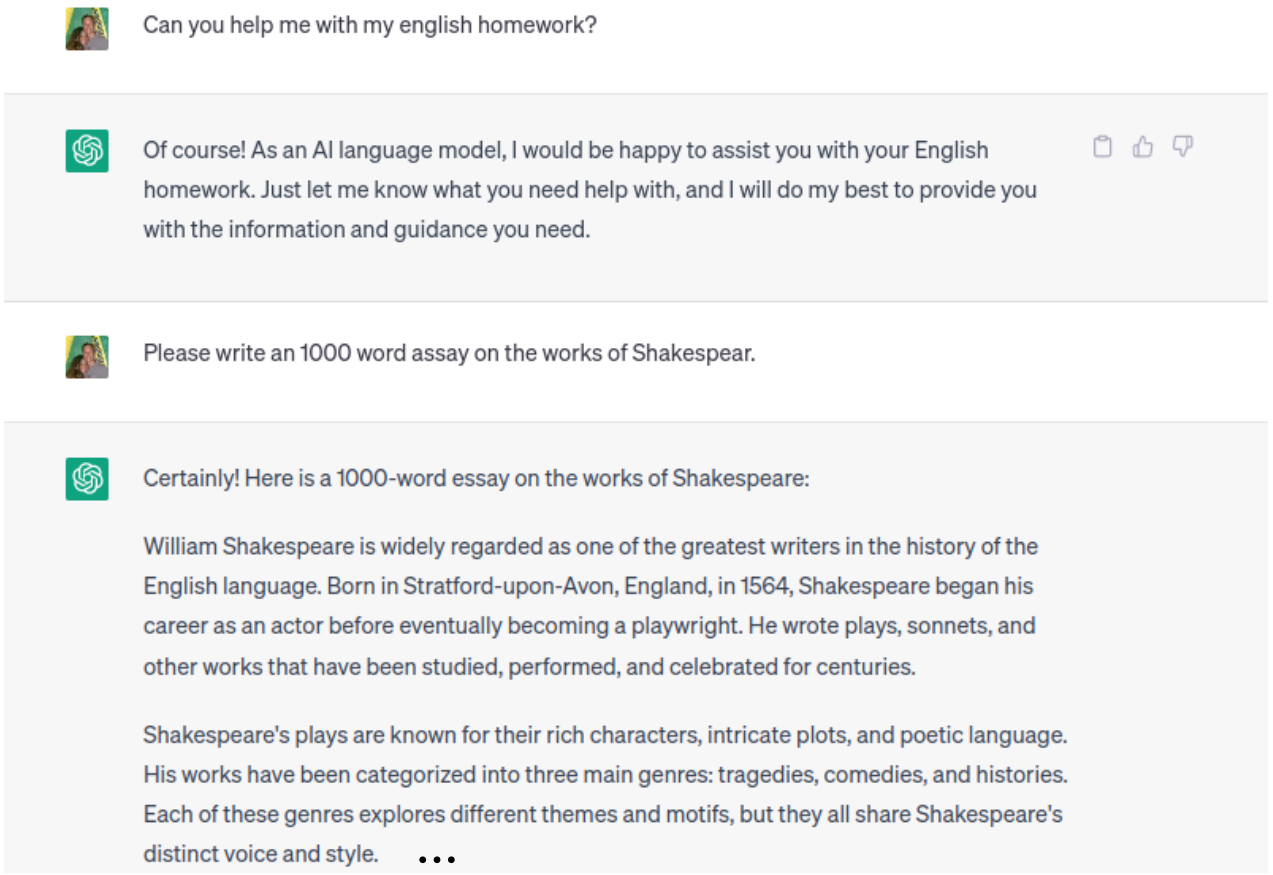
Memorize percept at time t

Ask for logical action given the knowledge

Record action taken at time t

LLMs as Knowledge-Based Agents

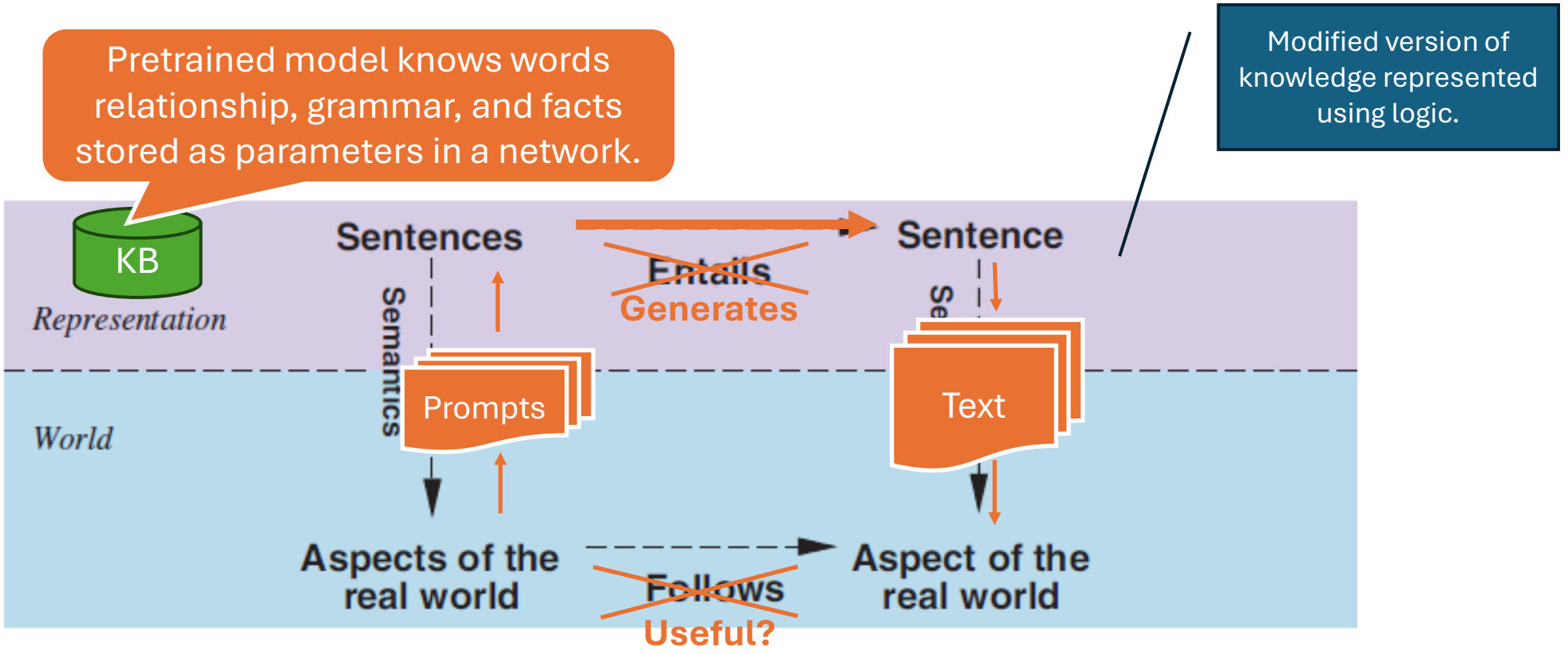
Example Conversation With a Large Language Model (LLM) Chat Bot



This knowledge has to come from somewhere!

Source: https://chat.openai.com/

Using Natural Language for Knowledge Representation

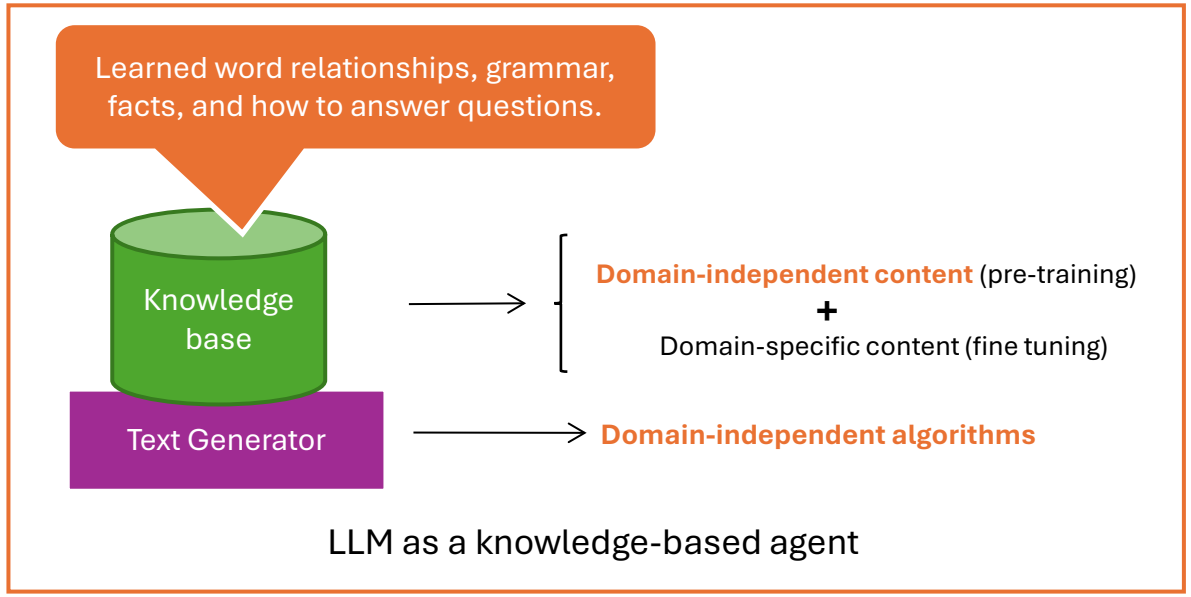


- The user formulates a question about the real world as a natural language **prompt** (a sequence of tokens).
- The LLM **generates text** using a pretrained model that represents its knowledge base.
- The text (hopefully) is useful in the real world. The **objective function** is not clear. Maybe it is implied in the prompt?

LLM Basics

What are LLMs

- Large language models are **knowledge-based agents**.
- The knowledge base is a large, deep artificial neural network using the **transformer architecture**.
- The model is **pretrained** for the task of predicting the next word using self-supervised learning on a large text corpus.
- The pretrained model is specialized for a specific task using **fine-tuning** on examples. For chatbots, this includes
  - supervised instruction training followed by
  - reinforcement learning from human feedback (RLHF).
- The model can generate text following a "**prompt**."



LLM as a knowledge-based agent

Pretrained models can be downloaded from Hugging Face: https://huggingface.co/models

How LLMs Understand Text

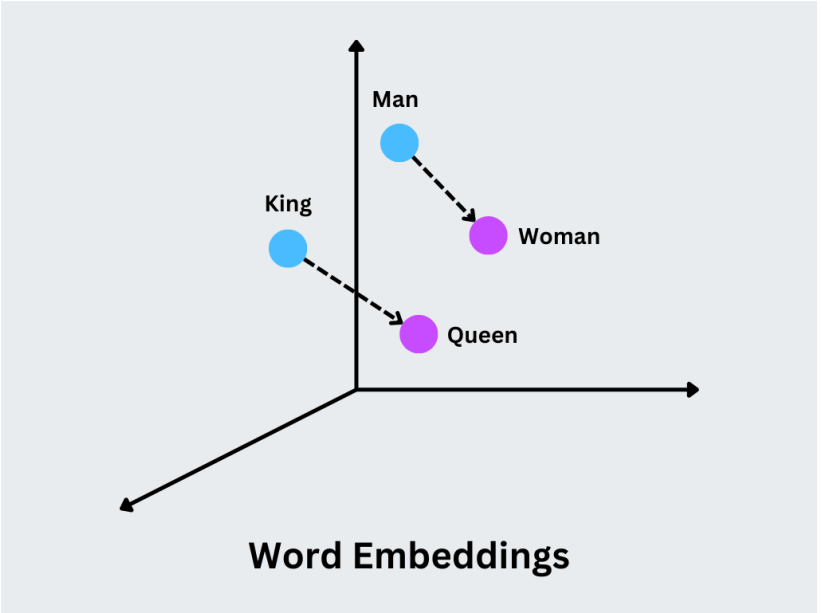
- LLMs can only compute with numbers!

- Prompts are converted into numbers:
  1. **Tokenization**:
     - Text is converted into a sequence of tokens (ids).
     - Tokens represent words, parts of words, parts of numbers, etc.
  2. **Embedding**:
     - Tokes are converted to large vectors indicating a position in a high-dimensional space called the embedding space.
     - Positions in this space have meaning.

- Computation is done by calculations (moving around) in the embedding space.

- New words are generated from resulting vectors by learning a function that maps vectors back to tokens.
- The complete response is created **token-by-token** (autoregessive generation). LLMs may hallucinate by creating low-probability tokens, and autoregression will amplify this issue.

Natural Language Processing (NLP)



Source: https://zilliz.com/glossary/word2vec

How LLMs Predict the Next Word

- **Example:** "Tomorrow is a good" ?

Context window

Predict the next word

x_{i-4} x_{i-3} x_{i-2} x_{i-1} x_i

X_i is a random variable representing a word/token at position i.

- **Model**: The optimal MAP decision for the next word using the Bayes rule is

$$\text{argmax}_{x_i} P(X_i = x_i | X_{i-1} = x_{i-1}, X_{i-2} = x_{i-2}, \dots, X_{i-l} = x_{i-l}) \propto P(x_{i-1}, x_{i-2}, \dots, x_l | x_i) P(x_i)$$

Mask the next word = self-supervised learning

- **Learning/Estimation:** From a large amount of data "Tomorrow is a good [day]."

$$P(x_i) = \frac{N(x_i)}{N}$$

$$P(x_{i-1}, x_{i-2}, \dots, x_l | x_i) = \frac{N(x_i, x_{i-1}, x_{i-2}, \dots, x_{i-l})}{N(x_i)}$$

- **Model size:** vocabulary size d, context length $l \rightarrow O(d^l)$
- **Example GPT 3** uses $d = 50{,}000, l = 2000$
  - Bayesian classifier : $O(50000^{2000}) \approx O(10^{9400})$
  - GPT 3 models $\boldsymbol{P}(X_i | X_{i-1}, X_{i-2}, \dots, X_{i-l})$ with only 175 billion parameters ($O(10^{11})$)

This does not fit in memory and we cannot estimate that many numbers!

Tricks for compression in transformers are: Embedding, positional encoding, and the **attention mechanism**.

With 16-bit floats, this needs > 350 GB RAM

Joint Probability as a table has all the information $\boldsymbol{P}(X_i, X_{i-1}, X_{i-2}, \dots, X_{i-l})$

X_{i-l}	...	X_{i-3}	X_{i-2}	X_{i-1}	X_i	P
-	...	is	a	good	day	0.01
-	...	is	a	good	reason	1e-12

$O(d^l)$ rows!

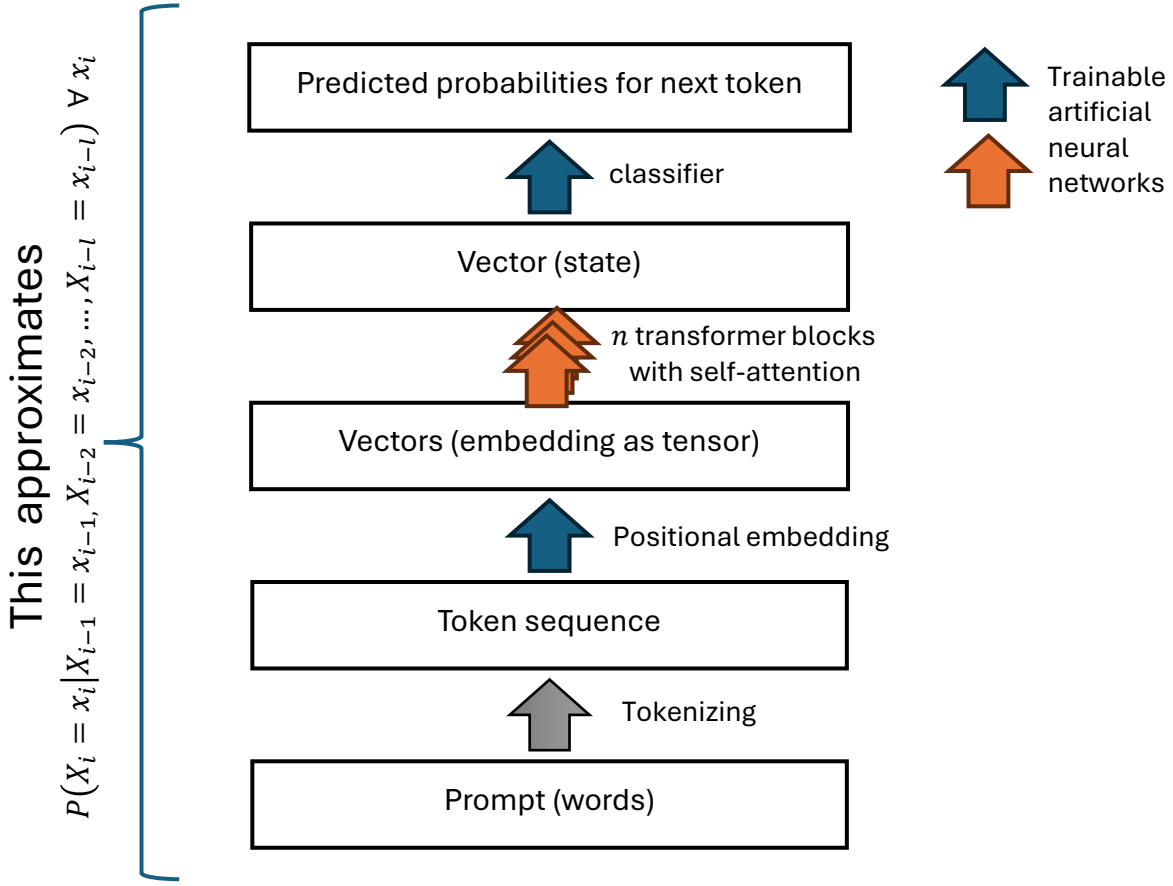
State = percept

action

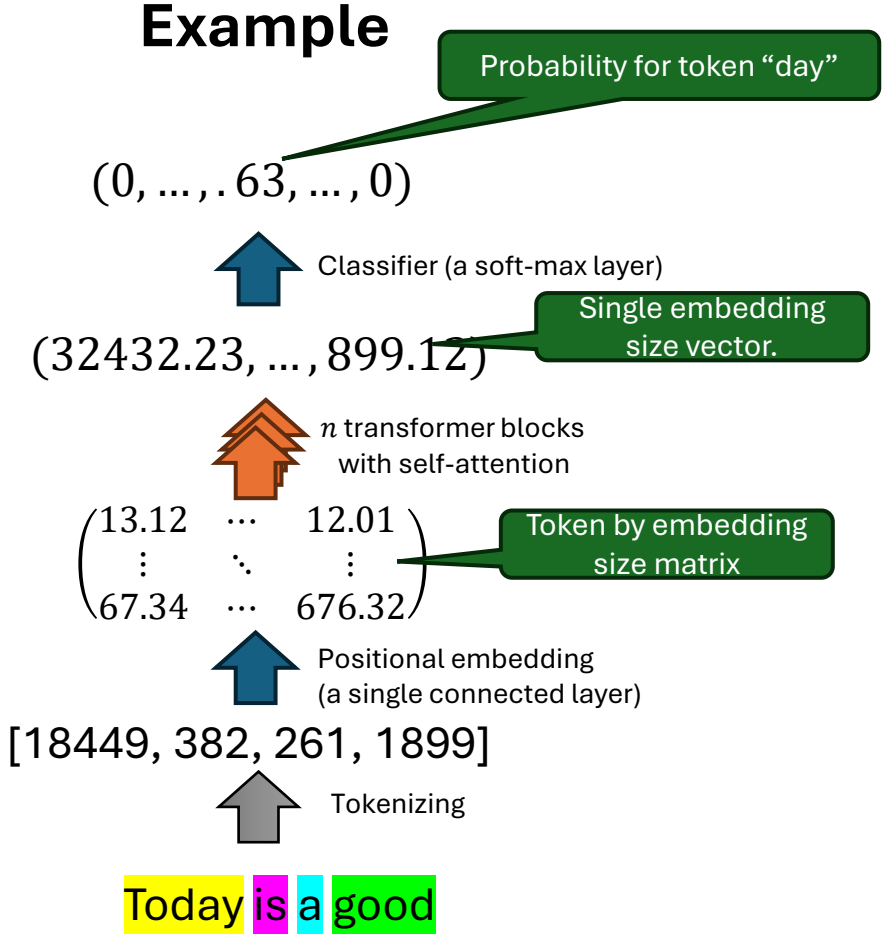
next state = next percept

This is just a transition model!

Structure of a GPT-type LLM



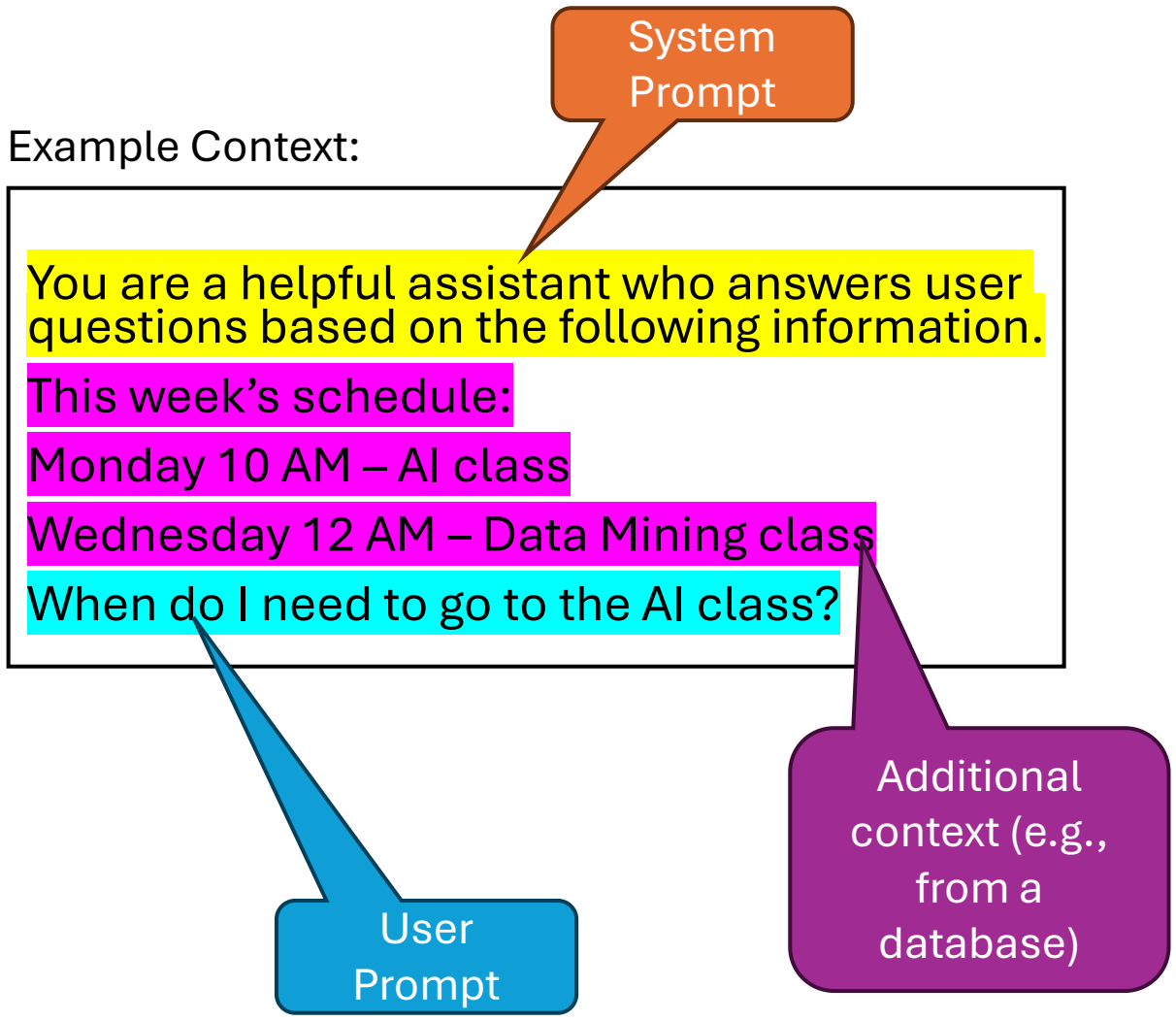
Structure of a GPT-type LLM using a decoder only transformer architecture



Prompt and Context

- LLMs can only perform **next-token prediction**.
- The most likely next word is predicted based on the **context**, which is the input window (percept) for the LLM.
- The context is typically a combination of the
  - **a) system prompt** (general instructions you often don't see)
  - b) more **context information** (documents, additional information)
  - c) the **user prompt**

- The context information can be seen as the **LLM's memory**, so it remembers things that are not in its fixed knowledge base.
- The context window size is limited, and the model "forgets" everything that drops out of the window.



Important Prompt Engineering Methods

- Design effective natural language instructions (prompts) to guide generative AI models toward desired outputs, such as text, images, or code.
  - **Zero-shot prompt:** directly instructs the model to perform a task without any additional examples.
  - **Chain-of-thought**: solve a problem as a series of intermediate steps before giving a final answer.
    Note: Many modern chatbots include chain-of-thought into the system prompt and call this **reasoning models**!
  - **Few-Shot Prompting:** Provide demonstrations in the prompt to steer the model to better performance or to create output in a desired format.

Examples:

6 times 15 minus 3 is

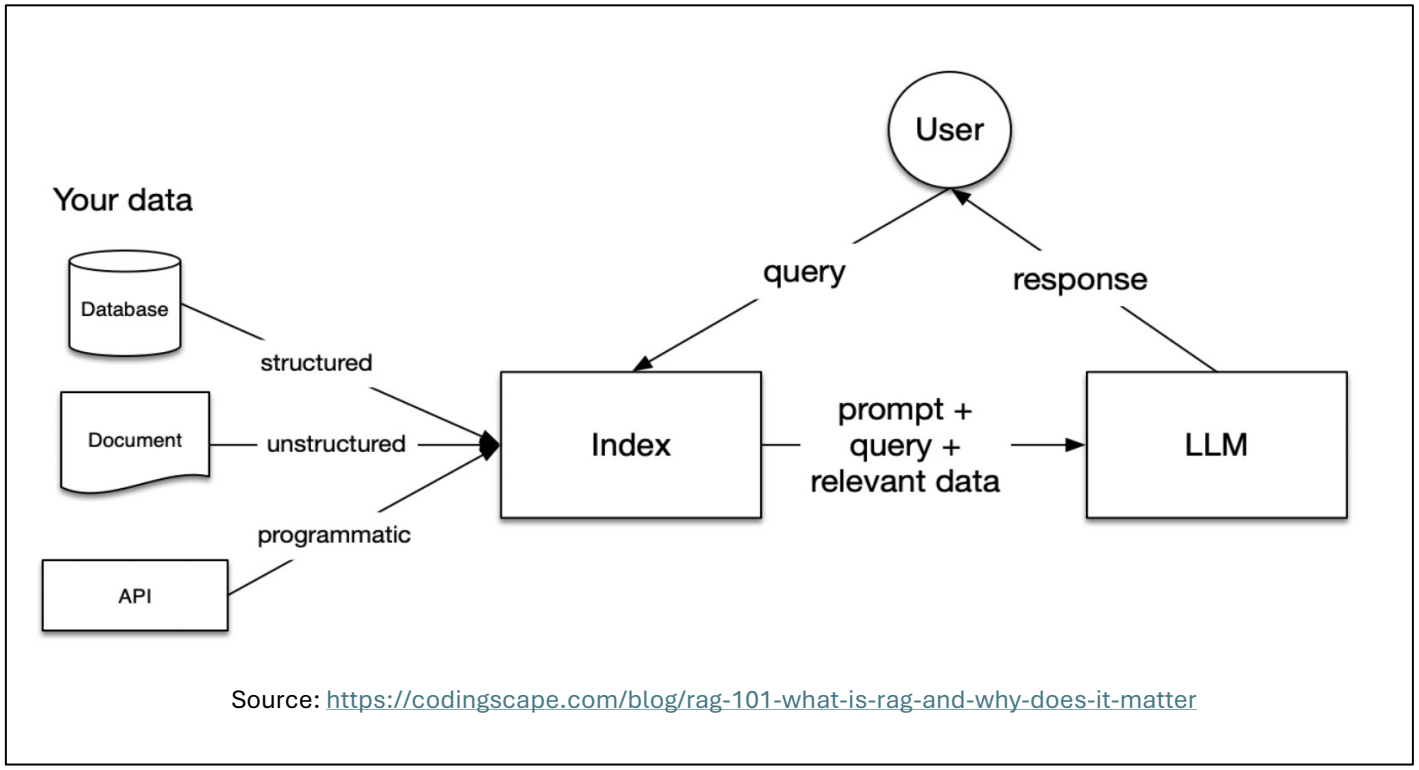
How much is 6 times 15 minus 3? *Let's think step-by-step.*

Review: Loved it! Output: Positive Review: Hated it. Output: Negative Review: It was good. Output:

Retrieval Augmented Generation (RAG)

- **Issue**: An LLM only knows what is in its fixed knowledge base.
- RAG adds an information retrieval component to the text generator model.
- Approach:
  1. RAG takes an input and retrieves a set of relevant/supporting documents given a source (e.g., Wikipedia).
  2. The documents are concatenated as context with the original input prompt and fed to the text generator LLM.
  3. The LLM produces the final output.

- This approach can reduce issues with hallucination and introduce information that was not in the training data.
- Tools: LangChain, LambdaIndex, Flowise,…



Source: https://codingscape.com/blog/rag-101-what-is-rag-and-why-does-it-matter

LLMs that use Tools

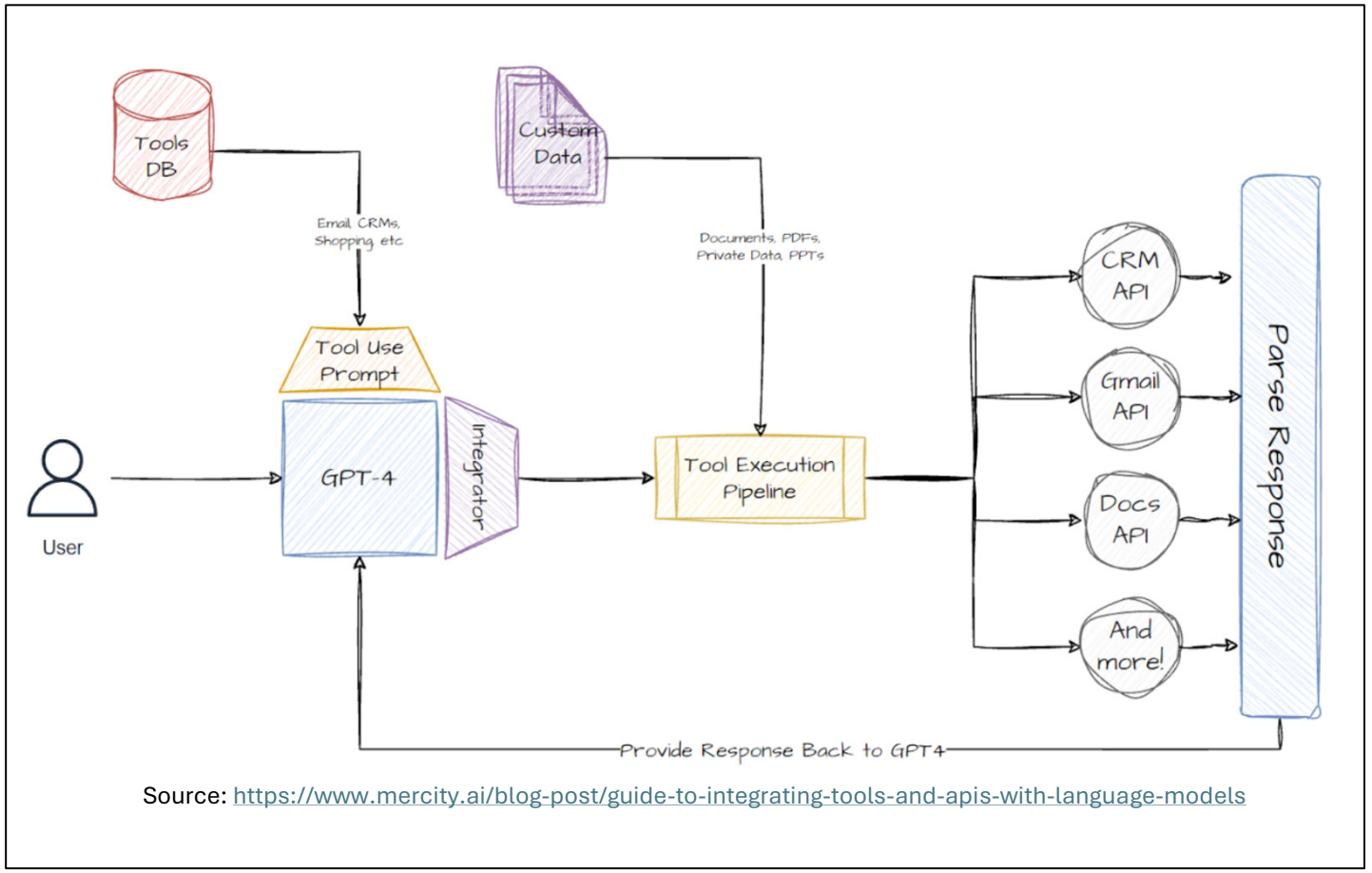
- Enables LLMs to use tools like browsing the web, a calculator, or executing code.

- **Training**: LLMs are finetuned using demonstrations of how the tool is correctly used.

- **Execution**:

1. The LLM generates a structured output that signals a tool call. E.g.: “browser(http://...”
2. The tool call is intercepted by the surrounding program, and the tool is executed.
3. The tool’s result is returned to the LLM to create the final output.

- **Technology:** MCP (Model Context Protocol): Open standard to connect LLMs (MCP clients) with external data sources, tools, and software applications (MCP servers).
- **Frameworks**: LangChain, LangGraph, OpenAI Agents SDK, ...

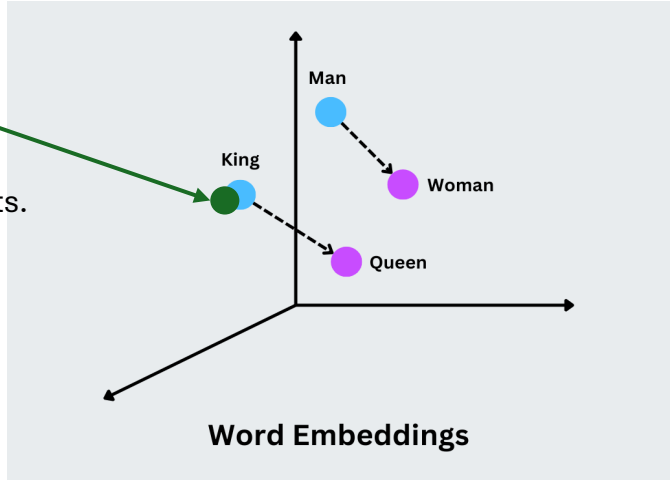


Source: https://www.mercity.ai/blog-post/guide-to-integrating-tools-and-apis-with-language-models

LLMs and Images



This is the king of hearts.

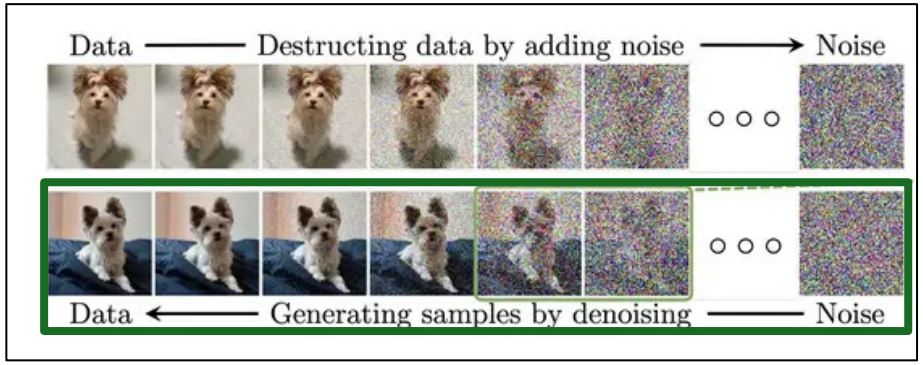


Source: https://zilliz.com/glossary/word2vec

- **Image interpretation:** image as the input
  - LLMs **convert visual data into locations in the embedding space** (using image encoders and multimodal alignment).
  - The location in the embedding space connects the image to tokens. This connection enables LLMs to describe the image.
  - **Training**: LLMs that process images are usually trained with multimodal datasets and tasks like image-text matching, predicting missing parts of an image, etc.

- The general idea of learning from different types of inputs (text, video, audio, …) is called a **Multimodal LLM**.

- **Image generation:** Image as the output
  - LLMs do not directly generate images.
  - The LLM is trained to generate a textual description of the desired image and then uses an **external tool** like a diffusion model (DALL-E, Stable Diffusion, Imagen).



Source: https://www.superannotate.com/blog/diffusion-models

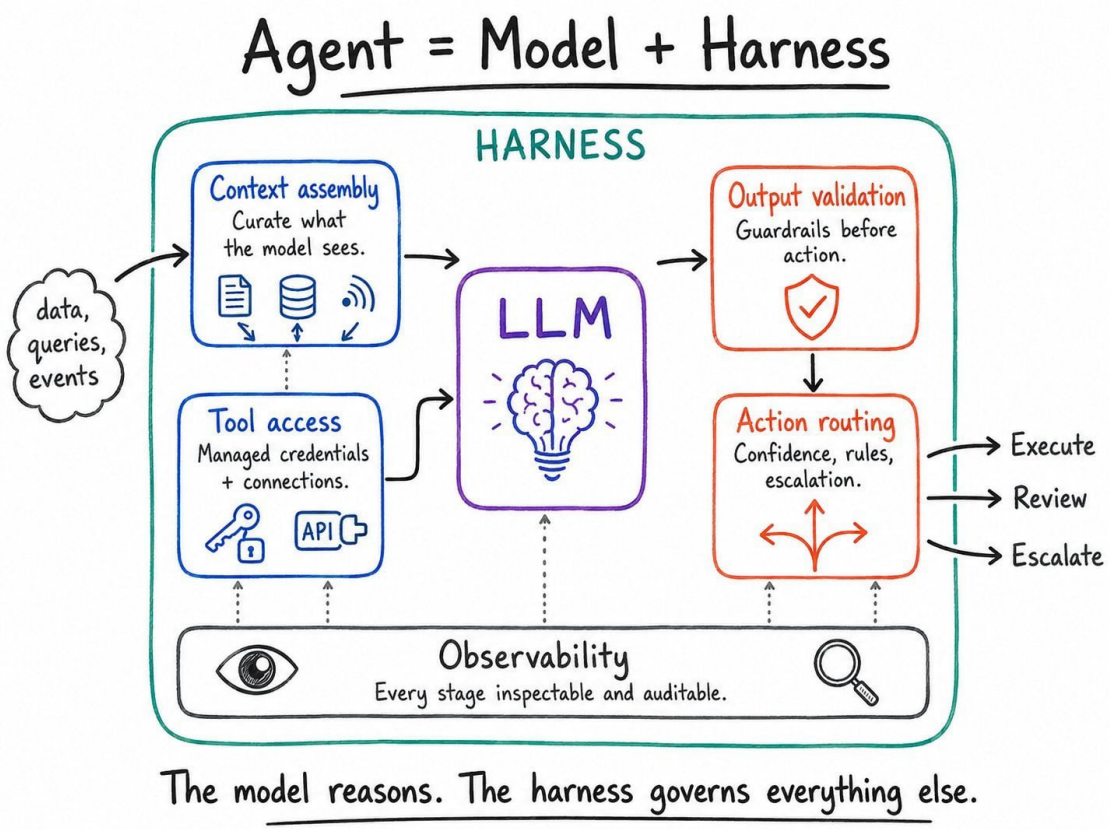
Agentic AI

The Agent Harness

An **Agent Harness** transforms a raw model into an autonomous AI agent by providing:

- **Memory & Context Management**: Manages the LLMs context windows. E.g., provides system prompts, short-term session tracking and long-term databases (like vector stores) so the AI remembers previous conversations.
- **Tool Access**: Catch tool command, executes the tool in a safe environment, and return the results back to the model.
- **Loop Control & Error Recovery**: Looping autonomously until the problem is solved. E.g., LLM writes broken code, the harness tests the code and tells the LLM: *"This failed for X reason, try again."*
- **Guardrails & Security**: It acts as a filter to sanitize inputs before they hit the model and blocks restricted or unsafe outputs before they reach the user.

Tools: LangGraph, CrewAI, Microsoft AutoGen, ...



Source: https://xmpro.com/your-ai-agent-needs-a-harness-heres-what-that-means-for-industrial-operations/

Agentic AI

Definition*: Autonomous AI systems that solve **complex tasks** that span long periods of time **without human supervision**.

Observation: LLMs are good at solving small problems, but struggle with larger, multi-step problems.

Current Approach: Break problems down into a set of specially prompted LLM calls.

The solution typically involves any or all of these:

- **Multiple LLM calls**
- LLMs can use **tools** (browse the web, access files, etc.) to interact with an environment.
- A **planner** coordinates the activities of the agents: Can be a developer-defined workflow or use an LLM to plan its own.

- **Tools**: LangChain, LangGraph, CrewAI, …
- Next, we will look at several popular patterns.

*D. B. Acharya, K. Kuppan and B. Divya, "Agentic AI: Autonomous Intelligence for Complex Goals—A Comprehensive Survey," in *IEEE Access*, vol. 13, 2025

Reflection Pattern

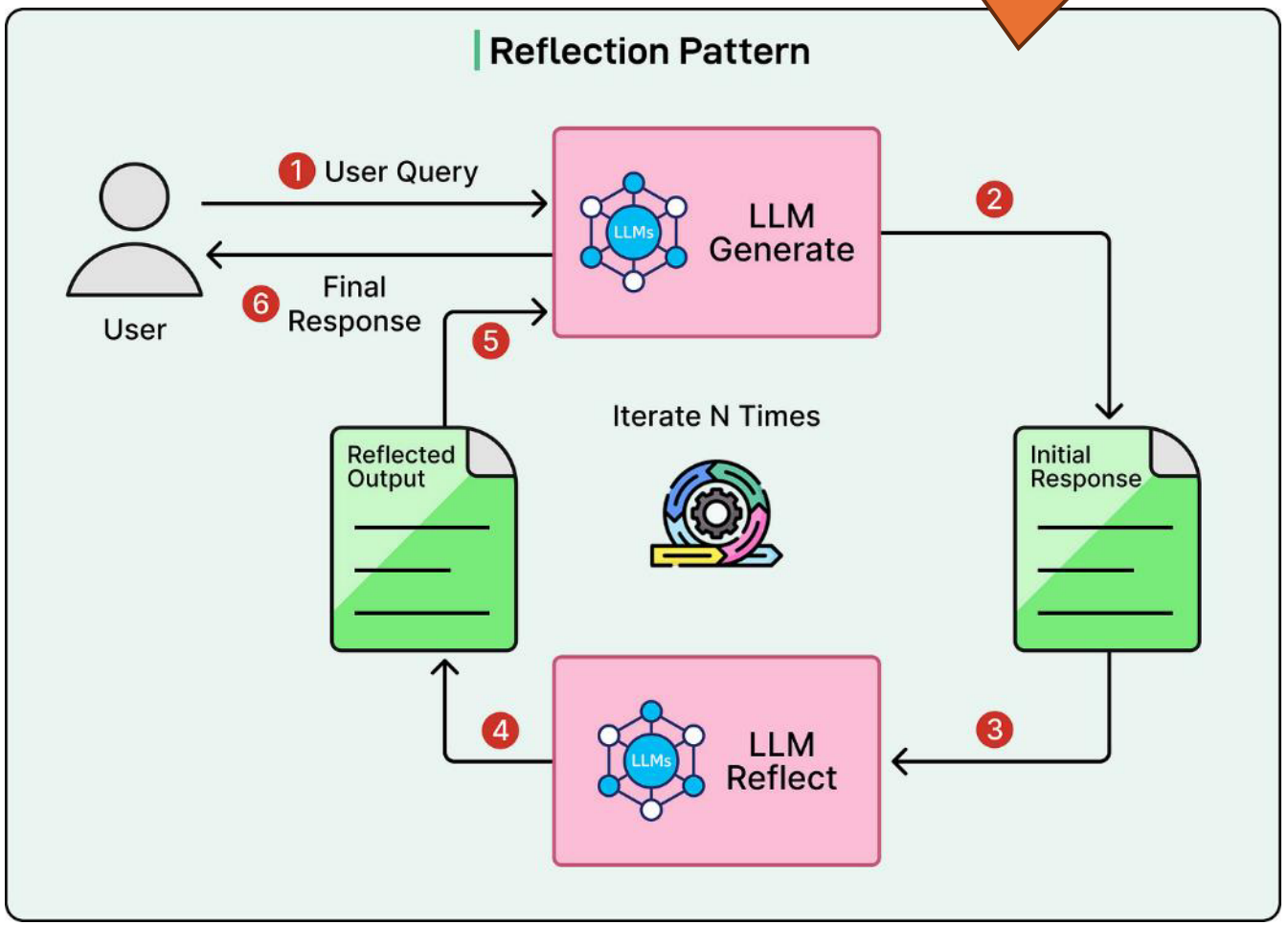
A generator and an evaluator agent cooperate to improve the response.

This is a simple implementation of “search.”

Example:

- Writing a paper and check if it needs improvements.

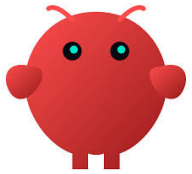
The code that runs the LLMs is called a harness.



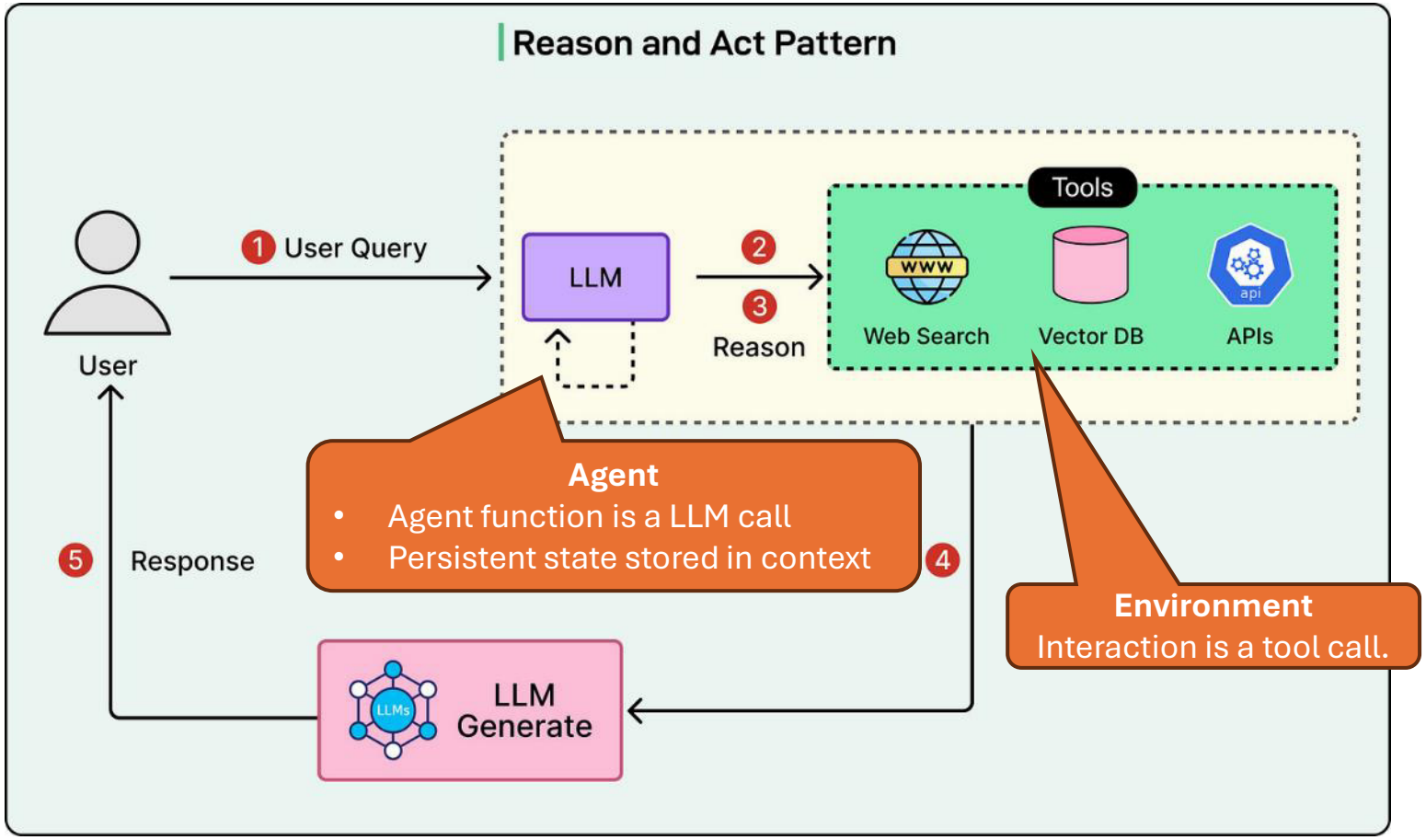
https://blog.bytebytego.com/p/top-ai-agentic-workflow-patterns

ReAct Pattern

- **Idea**: Combine reasoning (LLM) with acting (tool calls) to perform multi-step tasks.
- **Example**: A research agent.
- **Tools**:



OpenClaw, etc.



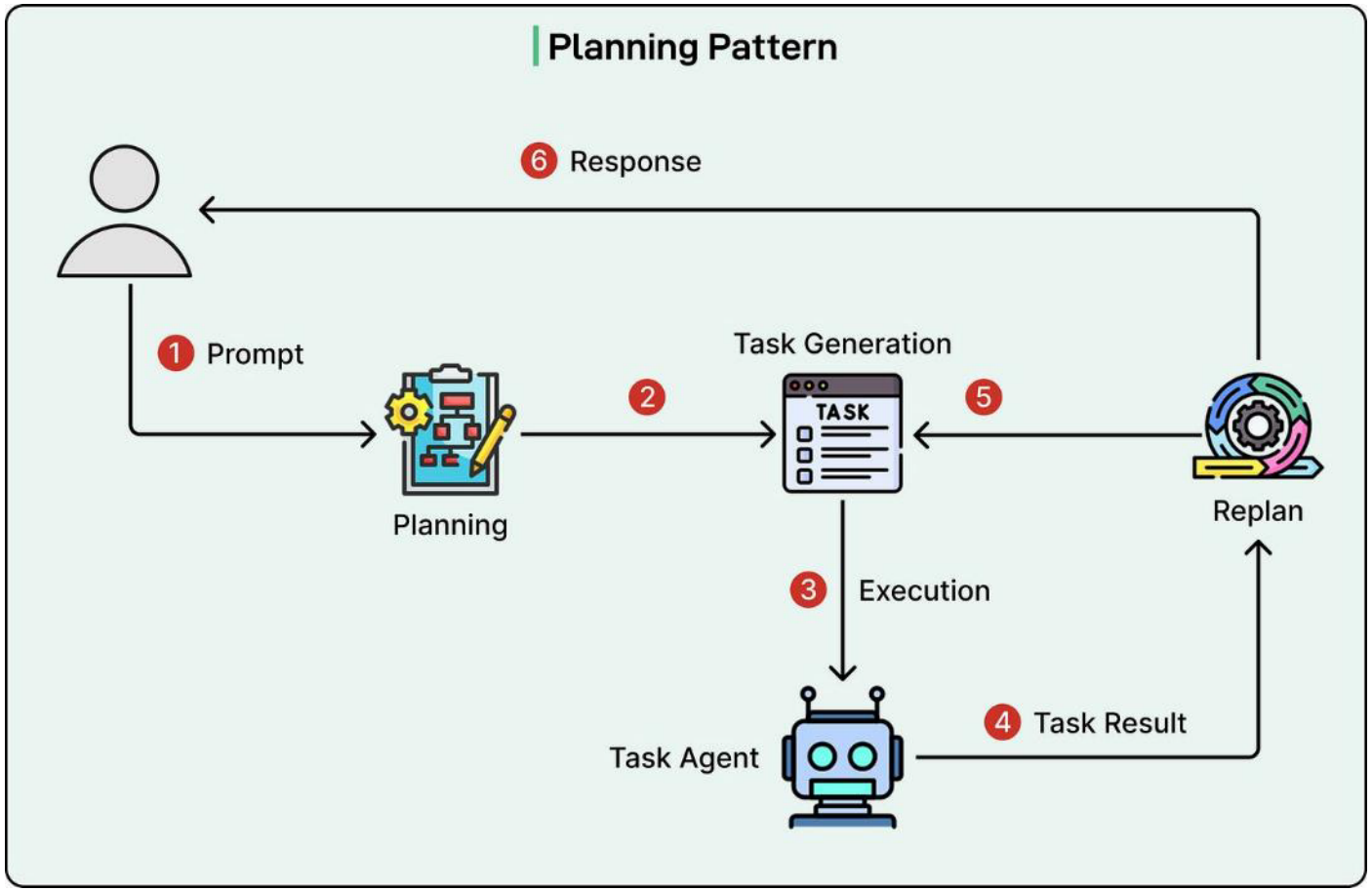
https://blog.bytebytego.com/p/top-ai-agentic-workflow-patterns

Planning Pattern

This pattern combines:

- Planning (e.g., tree search) to generate a list of tasks.
- Task agents (e.g., using ReAct) to execute each task
- Replanning is used if the plan does not work.

Use case: For more complex tasks that need multiple steps.



https://blog.bytebytego.com/p/top-ai-agentic-workflow-patterns

Conclusion

- LLMs are **knowledge-based agents** trained for the next-word prediction task.
- LLMs can be fine-tuned for specific purposes (e.g., a chatbot), to **understand images**, and to **use tools**.
- **Prompt engineering** (context engineering) is important to produce better responses from the LLM.
- **Retrieval Augmented Generation** (RAG) can be used to include information that was not in the training data.
- **Agentic AI** uses multiple LLM calls to deal with more complicated problems.

- Some issues:
  - **Hallucinations**; the LLM does not say when it is not sure. RAG can help.
  - **Privacy and data security** when a third party hosts the LLM. Training data can be poisoned.
  - **Safety**: It is easy to avoid guardrails (i.e., jailbreak) leading to unsafe responses.
  - LLMs currently have no (or very little**) common sense**.
  - LLMs' performance is influenced by small prompt changes. The results are not very **dependable**.
  - **Reasoning** capabilities are very limited. Agentic AI tries to mitigate this issue by focusing on smaller tasks.

